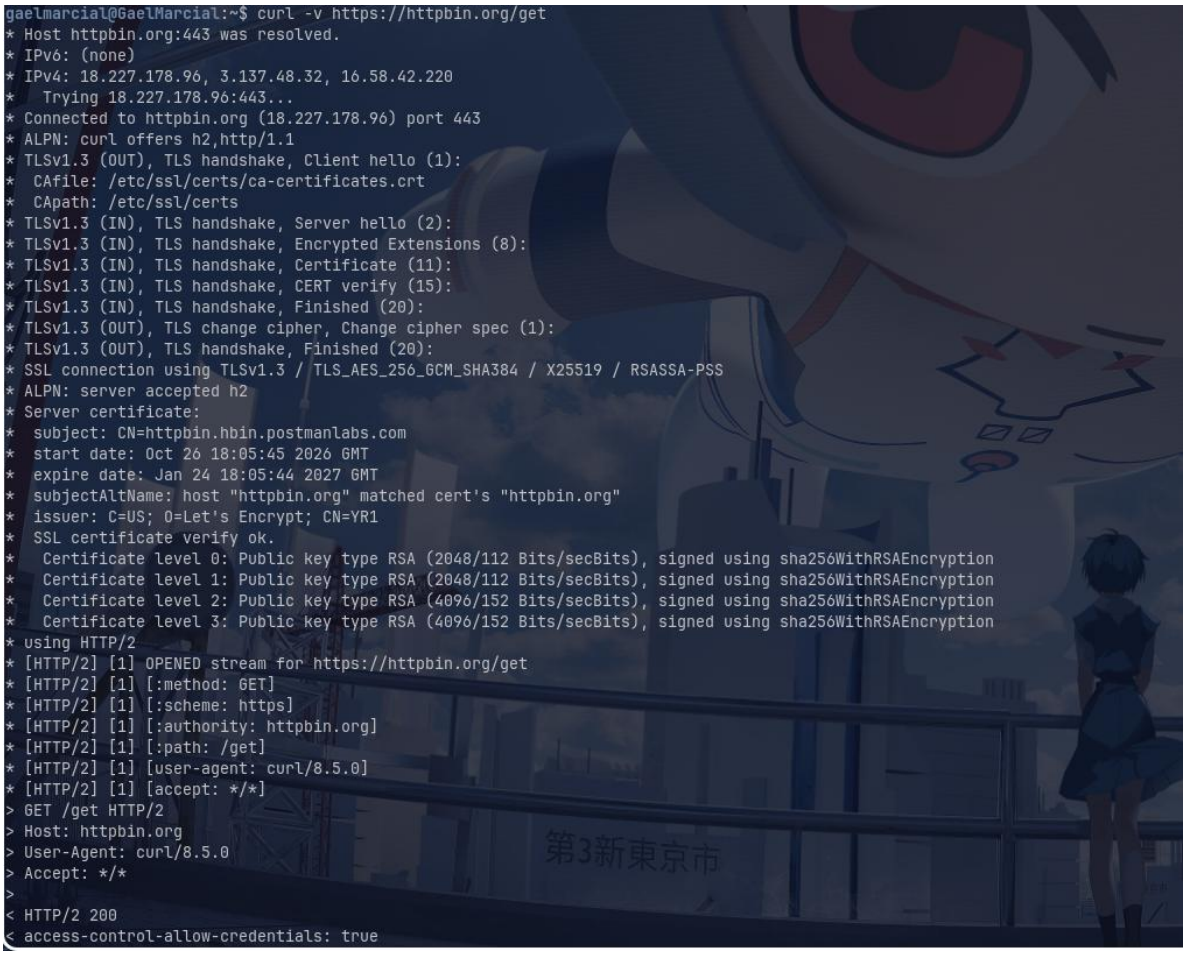
Jorge Gael Hernández Marcial

Ejercicio 1 · Un 200 a mano • Con curl -v pidan https://httpbin.org/get. Identifiquen en la salida: la línea de solicitud que ustedes generaron, el User-Agent de curl y la línea de estado 200.





Jorge Gael Hernández Marcial

Línea de solicitud: > GET /get HTTP/2.

User-Agent: > User-Agent: curl/8.5.0.

Línea de estado: < HTTP/2 200

Ejercicio 2 · Un 404 provocado • Pidan una ruta que no existe: curl -v https://httpbin.org/noexiste. Registren la línea de estado y expliquen por qué es un error 4xx y no 5xx.



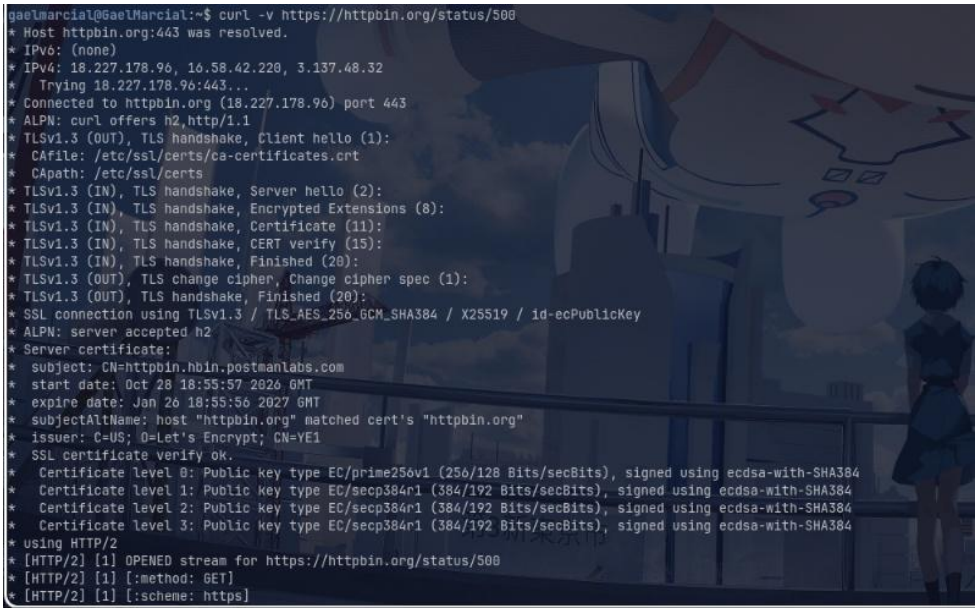
Línea de estado: El servidor devolvió la cabecera < HTTP/2 404.

Cuerpo de la respuesta: Se recibió un documento HTML con el título "404 Not Found" y el mensaje indicando que la URL solicitada no se encontró en el servidor.

Jorge Gael Hernández Marcial

Pertenece a la familia de errores 4xx porque la falla es responsabilidad del cliente, al haber construido una petición hacia una ruta (/noexiste) inválida o inexistente. El servidor funcionó correctamente y pudo procesar la solicitud para informar la ausencia del recurso, por lo que no clasifica como un error interno del servidor (5xx).

Ejercicio 3 · Un 500 bajo pedido • Pidan https://httpbin.org/status/500: el servidor simula una falla interna. Registren la respuesta y expliquen la diferencia de responsabilidad frente al 404.





La línea de estado que obtuve fue HTTP/2 500. La diferencia de responsabilidad frente al 404 es total: en este caso, la petición que envié como cliente estaba bien estructurada y era válida, pero fue el servidor el que tuvo una falla interna (en este caso, simulada) que le impidió generar una respuesta exitosa. La responsabilidad recae completamente del lado del servidor.n 000000000

Ejercicio 4 · Métodos distintos de GET • Envíen un POST con cuerpo a https://httpbin.org/post (como en el ejemplo de la sección 5) y un HEAD a https://httpbin.org/get (curl -I). Anoten qué cambió en la respuesta de cada uno.



Al enviar la petición POST incluyéndole datos, el servidor me respondió con un cuerpo en formato JSON donde refleja la información exacta que le envié en el campo "data": "apodo=ana", demostrando que la recibió y la procesó. En cambio, al utilizar el método HEAD (curl -I), el servidor contestó únicamente con los encabezados HTTP (mostrando un estado 200 OK y metadatos como el content-length: 202), pero omitió por completo el cuerpo de la respuesta. Esto comprueba que HEAD sirve para consultar el estado de un recurso y sus encabezados ahorrando ancho de banda.

Ejercicio 5 · La petición literal con netcat 2 Programación en Red · NRC 19141 · Unidad IV • Escriban la petición con printf + nc como en la sección 5. Comparen: ¿qué encabezados agrega curl por su cuenta que ustedes no escribieron a mano?



En esta petición manual con netcat, solo declaré explícitamente el encabezado Host: httpbin.org. Al observar la respuesta JSON, el servidor confirma que únicamente recibió ese encabezado. A diferencia de curl, que inyecta automáticamente encabezados adicionales como User-Agent y Accept por su cuenta, netcat transmite a la red de forma literal y estricta únicamente lo que escribiste en el comando printf.

Diferencia entre 401 y 403 Si un recurso protegido se pide sin credenciales, se espera un código 401 Unauthorized. La diferencia radica en que el 401 significa que el servidor no sabe quién eres porque falta autenticación (no enviaste credenciales o son incorrectas). Por otro lado, un 403 Forbidden ocurre cuando el servidor sí te ha autenticado y sabe quién eres, pero tus credenciales no tienen los privilegios o permisos necesarios para acceder a ese recurso específico (falta de autorización).

Manejo del código 500 en un sistema real Para evitar devolver un "500 pelado" (que a menudo expone la traza de la pila o *stack trace* con información sensible del servidor), el programador debe encapsular el código susceptible a fallas dentro de bloques try/catch. Al capturar la excepción, el servidor debe registrar el error real en un archivo de bitácora (log) interno para que los desarrolladores puedan depurarlo, y devolverle al cliente una respuesta HTTP estructurada y controlada, típicamente un JSON con un mensaje amigable como {"error": "Servicio temporalmente no disponible"}.

Servicio LA FILA como API web Si el servicio de cola de mensajes de la Unidad I se reescribiera como una API REST:

- Para formarse en la fila (insertar un elemento), el cliente usaría el método POST hacia una ruta como /fila, y el servidor respondería con un código 201 Created.

- Para atender al siguiente en la fila (sacar un elemento), el cliente usaría el método DELETE (o un POST a /fila/atender), y el servidor respondería con un código 200 OK si devuelve los datos del cliente atendido, o 204 No Content si la acción fue exitosa pero no hay datos adicionales que mostrar.